

Informe del taller Optiruta

Elaborado por:

Jhonatan Mauricio Trujillo Parra ID: 948747

Jonatan Stiven Fernandez Vargas ID: 922509

Presentado a:

Mg. Esteban Ernesto Morales Castro

NRC-10-81402

Corporación Universitaria Minuto de Dios

Ingeniería en sistemas

2026

Tabla de contenido

INTRODUCCION	5
1. DEFINICION DEL PROBLEMA.....	6
1.1 Descripción.....	6
1.2 Instancia trabajada.....	6
1.3 Restricciones.....	7
2. Dataset y fuentes de datos	7
2.1 Extracción de datos geográficos.....	7
2.2 Estructura del dataset.....	11
2.3 Matriz de distancias.....	12
3. algoritmos implementados.....	12
3.1 Insertion Sort	12
Descripción	12
Complejidad.....	12
Justificación	12
3.2 Algoritmo Voraz (Greedy).	13
Descripción	13
Complejidad.....	13
Resultado obtenido	13
3.3 Programación Dinámica (Problema de la Mochila 0/1).	13

Descripción	13
Complejidad.....	14
Resultados obtenidos	14
3.4 Backtracking con Poda	14
Descripción	14
Función de poda.....	15
Complejidad.....	15
Resultados obtenidos	15
4. Análisis de Complejidad.....	16
4.1 Comparativa experimental por tamaño de instancia.....	16
4.2 Análisis del efecto de la poda.	16
5. ARQUITECTURA IMPLEMENTADA AL PROYECTO.....	17
5.1 Estructura de módulos.	17
extractor.py	18
5.2 Flujo de ejecución.....	22
• Paso 1.....	22
• Paso 2.....	22
• Paso 3.....	22
• Paso 3B.....	22
• Paso 4.....	22

• Paso 5.....	22
main.py	23
6. Visualización Interactiva	27
6.1 Mapa interactivo (mapa.html).	27
6.2 Árbol de backtracking (index.html).....	28
CONCLUSION.....	30
REFERENCIAS	31

INTRODUCCION

El proyecto OPTIRUTA+ nace como respuesta a un problema real y cotidiano: la optimización de rutas de entrega para una empresa de mensajería en la ciudad de Ibagué. En el contexto de la logística moderna, resolver de manera eficiente la distribución de paquetes a múltiples clientes, respetando las restricciones de capacidad de los vehículos disponibles, representa uno de los retos computacionales más complejos e interesantes en el campo del diseño de algoritmos.

Este proyecto integra los paradigmas estudiados durante el semestre en una solución híbrida que combina múltiples técnicas algorítmicas. Se implementó un Algoritmo Voraz (Greedy) para generar una ruta inicial usando el criterio del vecino más cercano, Programación Dinámica para resolver el problema de la mochila y asignar paquetes a vehículos de forma óptima, un algoritmo de Backtracking con poda para explorar el espacio de soluciones y encontrar la ruta exacta, e Insertion Sort para ordenar los puntos de entrega por distancia al depósito.

El sistema fue desarrollado en Python, usando datos geográficos reales de Ibagué extraídos de OpenStreetMap mediante las librerías osmnx y NetworkX. Los resultados se presentan tanto en consola como en una visualización web interactiva que muestra el mapa real de Ibagué con las rutas generadas y la animación del proceso de backtracking.

1. DEFINICION DEL PROBLEMA

1.1 Descripción.

Una empresa de mensajería necesita entregar paquetes a N clientes distribuidos por la ciudad. Para esto dispone de una flota de vehículos con capacidad de carga limitada. El objetivo es encontrar la ruta que minimice la distancia total recorrida, respetando en todo momento la capacidad máxima de cada vehículo.

1.2 Instancia trabajada.

Para el desarrollo del proyecto se trabajó con 8 clientes ($N=8$) como instancia de validación, extraídos del grafo real de calles de Ibagué. El dataset incluye tres tipos de vehículos con características distintas:

Vehículo	Peso Max (kg)	Volumen (cm)	Kilometraje
Moto	35	40×40×50	250 km
Carro	500	120×100×150	600 km
Camión	8000	500×300×800	1200 km

```
data > {} vehiculos.json > ...  
You, la semana pasada | 1 author (You)  
1 {  
2   "vehiculos": [  
3     {  
4       "tipo": "moto",  
5       "volumen_max": [40.0, 40.0, 50.0],  
6       "peso_max_kg": 35.0,  
7       "kilometraje": 250.0  
8     },  
9     {  
10      "tipo": "carro",  
11      "volumen_max": [120.0, 100.0, 150.0],  
12      "peso_max_kg": 500.0,  
13      "kilometraje": 600.0  
14    },  
15    {  
16      "tipo": "camion",  
17      "volumen_max": [500.0, 300.0, 800.0],  
18      "peso_max_kg": 8000.0,  
19      "kilometraje": 1200.0  
20    }  
21  ]  
22 }
```

1.3 Restricciones.

- Cada vehículo no puede superar su capacidad máxima de peso.
- Todos los clientes deben ser visitados exactamente una vez.
- La ruta debe comenzar y terminar en el depósito (nodo 0).
- Las distancias entre nodos se calculan usando rutas reales de calles de Ibagué.

2. DATASET Y FUENTES DE DATOS

2.1 Extracción de datos geográficos.

El dataset fue construido combinando dos fuentes de datos reales:

- **osmnx**: Librería de Python que descarga el grafo completo de calles de Ibagué desde OpenStreetMap. El grafo obtenido contiene 9.048 nodos y 22.436 aristas, representando todas las intersecciones y vías de la ciudad.
- **Nominatim API**: Servicio de geocodificación de OpenStreetMap que permite buscar coordenadas reales de lugares por nombre. Se usó para extraer 15 destinos reales de Ibagué.

Los destinos reales incluidos en el dataset son puntos de referencia conocidos de la ciudad:

- **Centros comerciales**: Multicentro, La Estación
- **Hospitales**: Federico Lleras Acosta, San Francisco
- **Universidades**: Universidad del Tolima, Universidad de Ibagué
- **Parques y espacios públicos**: Parque Centenario, Plaza de Bolívar, Parque de la Música
- **Infraestructura**: Terminal de Transportes, Aeropuerto Perales
- **Barrios**: El Salado, Chapetón, Picaleña

```
data > extractor.py > extraer_clientes
You, la semana pasada | 1 author (You)
1 import osmnx as ox
2 import networkx as nx
3 import json
4 import random
5 import requests
6 import time
7
8 # --- CONFIGURACIÓN ---
9 CIUDAD = "Ibagué, Tolima, Colombia"
10
11 DESTINOS_FIJOS = [
12     "Centro Comercial Multicentro Ibagué",
13     "Centro Comercial La Estación Ibagué",
14     "Parque Centenario Ibagué",
15     "Plaza de Bolívar Ibagué",
16     "Terminal de Transportes Ibagué",
17     "Aeropuerto Perales Ibagué",
18     "Hospital Federico Lleras Acosta Ibagué",
19     "Universidad del Tolima Ibagué",
20     "Universidad de Ibagué",
21     "Estadio Manuel Murillo Toro Ibagué",
22     "Parque de la Música Ibagué",
23     "Hospital San Francisco Ibagué",
24     "Barrio El Salado Ibagué",
25     "Chapetón Ibagué",
26     "Picaleña Ibagué"
27 ]
28
29 PRODUCTOS = [
30     "Nevera", "Caja de frutas", "Televisor", "Colchón",
31     "Caja de libros", "Electrodoméstico", "Ropa empacada",
32     "Caja de medicamentos", "Computador", "Mueble desmontado",
33     "Caja de herramientas", "Juguetes", "Documentos",
34     "Caja de alimentos secos", "Botellón de agua", "Equipo deportivo", "Caja de productos electrónicos",
35     "Spray de limpieza", "Labial rojo", "Pala", "Bicicleta"
36 ]
37
```

```

38 # — 1. CARGAR GRAFO (osmnx) —
39 def cargar_grafo():
40     print("Descargando grafo de Ibagué...")
41     G = ox.graph_from_place(CIUDAD, network_type="drive")
42     print(f"Grafo cargado: {len(G.nodes())} nodos, {len(G.edges())} aristas")
43     return G
44
45 # — 2. BUSCAR DESTINOS (Nominatim) —
46 def buscar_destinos():
47     print("\nBuscando coordenadas de destinos con Nominatim...")
48     destinos = []
49
50     for nombre in DESTINOS_FIJOS:
51         url = "https://nominatim.openstreetmap.org/search"
52         params = {
53             "q": nombre,
54             "format": "json",
55             "countrycodes": "co",
56             "limit": 1
57         }
58         headers = {"User-Agent": "OptiroutaPlus/1.0"}
59         respuesta = requests.get(url, params=params, headers=headers)
60         resultados = respuesta.json()
61
62         if resultados:
63             r = resultados[0]
64             destinos.append({
65                 "nombre": nombre,
66                 "latitud": float(r["lat"]),
67                 "longitud": float(r["lon"])
68             })
69             print(f" ✓ {nombre}")
70         else:
71             print(f" X No encontrado: {nombre}")
72
73         time.sleep(1)
74
75     print(f"Destinos encontrados: {len(destinos)}")
76     return destinos

```

```

78 # — 3. PUENTE: coordenadas → nodos del grafo (osmnx) —
79 def destinos_a_nodos(G, destinos):
80     print("\nMapeando destinos al grafo de calles...")
81     destinos_con_nodo = []
82
83     for d in destinos:
84         nodo_id = ox.nearest_nodes(G, X=d["longitud"], Y=d["latitud"])
85         data = G.nodes[nodo_id]
86         destinos_con_nodo.append({
87             "nombre": d["nombre"],
88             "latitud_real": d["latitud"],
89             "longitud_real": d["longitud"],
90             "node_id": nodo_id,
91             "latitud_nodo": data["y"],
92             "longitud_nodo": data["x"]
93         })
94     print(f" ✓ {d['nombre']} → nodo {nodo_id}")
95
96     return destinos_con_nodo

```

2.2 Estructura del dataset.

Cada cliente en el dataset contiene la siguiente información:

Campo	Tipo	Descripción
node_id	int	ID del nodo en el grafo OSM
latitud / longitud	float	Coordenadas del punto de origen
peso_kg	float	Peso del paquete (1–50 kg)
volumen	list[float]	Dimensiones del paquete (x, y, z) en cm
producto	str	Nombre del producto a entregar
destino	str	Nombre del lugar de destino
destino_latitud / longitud	float	Coordenadas del destino real

2.3 Matriz de distancias.

Se calculó una matriz de distancias reales de 8×8 usando el algoritmo de camino más corto de Dijkstra implementado en NetworkX (`nx.shortest_path_length` con `weight='length'`). Esto garantiza que las distancias reflejan los recorridos reales por las calles de Ibagué, no distancias euclidianas.

3. ALGORITMOS IMPLEMENTADOS

3.1 Insertion Sort

Descripción

Se implementó el algoritmo Insertion Sort para ordenar los clientes de menor a mayor distancia respecto al depósito. Este algoritmo toma cada elemento y lo inserta en la posición correcta dentro de la parte ya ordenada de la lista, desplazando los elementos mayores hacia la derecha.

Complejidad

- Temporal: $O(n^2)$ en el caso promedio y peor caso.
- Espacial: $O(1)$ — ordenamiento en sitio, no requiere memoria adicional.

Justificación

Para $N=8$ clientes, Insertion Sort es completamente adecuado. Su implementación es sencilla, transparente y permite demostrar el proceso de ordenamiento de manera

clara. El resultado se usa como información auxiliar para priorizar qué clientes visitar primero.

3.2 Algoritmo Voraz (Greedy).

Descripción

El algoritmo voraz implementa la heurística del vecino más cercano para construir una ruta inicial. Partiendo del depósito (nodo 0), en cada paso selecciona el cliente no visitado más cercano al nodo actual. Esta decisión local óptima no garantiza la solución global óptima, pero genera una buena aproximación en tiempo polinomial.

Complejidad

- Temporal: $O(n^2)$ — para cada uno de los n nodos, busca el mínimo entre los $n-1$ restantes.
- Espacial: $O(n)$ — almacena la ruta y el vector de visitados.

Resultado obtenido

Para la instancia de 8 clientes, el algoritmo voraz generó una ruta con distancia total de 46.384,7 metros (46,38 km). Esta distancia se usó como límite superior inicial para el backtracking.

3.3 Programación Dinámica (Problema de la Mochila 0/1).

Descripción

Se implementó el clásico algoritmo de la mochila 0/1 mediante programación dinámica para resolver el subproblema de asignación de paquetes a cada vehículo. El

algoritmo construye una tabla $dp[i][w]$ donde cada celda representa el máximo peso transportable usando los primeros i paquetes con una capacidad de w kilogramos.

Regla de recurrencia: Si $peso[i] > w$: $dp[i][w] = dp[i-1][w]$. Si no: $dp[i][w] = \max(dp[i-1][w], dp[i-1][w - peso[i]] + peso[i])$.

Complejidad

- Temporal: $O(n \times C)$ — pseudo-polinomial, donde n es el número de paquetes y C la capacidad del vehículo.
- Espacial: $O(n \times C)$ — requiere la tabla completa para la reconstrucción de la solución.

Resultados obtenidos

Vehículo	Capacidad (kg)	Peso Cargado (kg)	Paquetes
Moto	35	35	2
Carro	500	236	8
Camión	8.000	236	8

3.4 Backtracking con Poda

Descripción

El algoritmo de backtracking es el núcleo del proyecto. Explora el espacio de todas las permutaciones posibles de rutas mediante una función recursiva que construye soluciones parciales y retrocede cuando detecta que no pueden llevar a una solución mejor que la actual.

Función de poda

Se implementaron dos condiciones de poda para reducir drásticamente el espacio de búsqueda:

- Poda por distancia: Si la distancia parcial acumulada ya supera la mejor solución conocida, se abandona esa rama sin explorarla.
- Poda por capacidad: Si el peso acumulado de los paquetes supera la capacidad del vehículo, se descarta esa asignación.

Complejidad

- Sin poda: $O(n!)$ — factorial en el número de clientes.
- Con poda: Significativamente menor en la práctica, aunque sigue siendo exponencial en el peor caso.

Resultados obtenidos

Vehículo	Voraz (m)	Backtracking (m)	Mejora (m)	Tiempo (s)
Moto	46.384,7	Sin ruta factible	—	0,031
Carro	46.384,7	40.211,11	6.173,59	0,262
Camión	46.384,7	40.211,11	6.173,59	0,244

La moto no encontró ruta factible porque su capacidad (35 kg) no es suficiente para cubrir todos los clientes de la instancia. El carro y el camión encontraron la misma ruta óptima al tener ambos capacidad suficiente.

4. ANÁLISIS DE COMPLEJIDAD

4.1 Comparativa experimental por tamaño de instancia.

N	Voraz (m)	Backtracking (m)	Mejora (m)	Tiempo BT
8	40.646,92	38.622,67	2.024,25	0,003 s
10	61.142,88	50.288,09	10.854,79	0,147 s
12	74.213,57	63.314,41	10.899,16	5,88 s
15	79.884,28	—	—	> 2 min

La tabla evidencia el comportamiento factorial del backtracking: al pasar de N=12 a N=15, el tiempo de ejecución supera los 2 minutos, confirmando que el algoritmo exacto no escala para instancias grandes. Esto justifica el uso del algoritmo voraz como solución heurística inicial y demuestra la importancia de la poda para mantener el backtracking viable en instancias medianas.

4.2 Análisis del efecto de la poda.

Para la instancia de N=8 clientes con el vehículo carro, el backtracking registró los siguientes contadores durante la exploración:

- Pasos explorados: 15.849
- Podas por distancia: Reducen ramas donde la distancia parcial ya supera la mejor solución.
- Podas por peso: Eliminan asignaciones que violan la capacidad del vehículo.
- Mejoras encontradas: Actualizaciones de la mejor ruta durante la exploración.

Sin poda, el número de permutaciones a explorar sería $7! = 5.040$ rutas completas, pero con todas sus expansiones intermedias el árbol completo contendría millones de nodos. La poda reduce este espacio en varios órdenes de magnitud.

5. ARQUITECTURA IMPLEMENTADA AL PROYECTO

5.1 Estructura de módulos.

Módulo	Descripción
data/extractor.py	Extrae datos reales de Ibagué usando osmnx y Nominatim. Genera clientes.json con la matriz de distancias.
modelos/cliente.py	Clase Cliente con atributos: nombre, coordenadas, peso, volumen, producto, origen y destino.
modelos/vehiculo.py	Clase Vehiculo con atributos: tipo, volumen_max, peso_max_kg, kilometraje.
algoritmos/ordenamiento.py	Implementa Insertion Sort para ordenar clientes por distancia al depósito.
algoritmos/voraz.py	Heurística del vecino más cercano. Genera ruta inicial en $O(n^2)$.
algoritmos/mochila_dp.py	Problema de la mochila 0/1 con DP. Asigna paquetes a vehículos.
algoritmos/backtracking.py	Búsqueda exacta con poda por distancia y peso. Registra cada paso.
main.py	Punto de entrada. Integra todos los módulos y exporta resultados.

Módulo	Descripción
visualizacion/index.html	Árbol de backtracking interactivo con Canvas y animación en tiempo real.
visualizacion/mapa.html	Mapa real de Ibagué con Leaflet.js. Muestra clientes, destinos, rutas y animación del backtracking.

extractor.py

```

You, la semana pasada | 1 author (You)
1  import osmnx as ox
2  import networkx as nx
3  import json
4  import random
5  import requests
6  import time
7
8  # — CONFIGURACIÓN —
9  CIUDAD = "Ibagué, Tolima, Colombia"
10
11  DESTINOS_FIJOS = [
12      "Centro Comercial Multicentro Ibagué",
13      "Centro Comercial La Estación Ibagué",
14      "Parque Centenario Ibagué",
15      "Plaza de Bolívar Ibagué",
16      "Terminal de Transportes Ibagué",
17      "Aeropuerto Perales Ibagué",
18      "Hospital Federico Lleras Acosta Ibagué",
19      "Universidad del Tolima Ibagué",
20      "Universidad de Ibagué",
21      "Estadio Manuel Murillo Toro Ibagué",
22      "Parque de la Música Ibagué",
23      "Hospital San Francisco Ibagué",
24      "Barrio El Salado Ibagué",
25      "Chapetón Ibagué",
26      "Picaleña Ibagué"
27  ]
28
29  PRODUCTOS = [
30      "Nevera", "Caja de frutas", "Televisor", "Colchón",
31      "Caja de libros", "Electrodoméstico", "Ropa empacada",
32      "Caja de medicamentos", "Computador", "Mueble desmontado",
33      "Caja de herramientas", "Juguetes", "Documentos",
34      "Caja de alimentos secos", "Botellón de agua", "Equipo deportivo", "Caja de productos electrónicos",
35      "Spray de limpieza", "Labial rojo", "Pala", "Bicicleta"
36  ]
37
  
```

```
38 # — 1. CARGAR GRAFO (osmnx) —————
39 def cargar_grafo():
40     print("Descargando grafo de Ibagué...")
41     G = ox.graph_from_place(CIUDAD, network_type="drive")
42     print(f"Grafo cargado: {len(G.nodes())} nodos, {len(G.edges())} aristas")
43     return G
44
45 # — 2. BUSCAR DESTINOS (Nominatim) —————
46 def buscar_destinos():
47     print("\nBuscando coordenadas de destinos con Nominatim...")
48     destinos = []
49
50     for nombre in DESTINOS_FIJOS:
51         url = "https://nominatim.openstreetmap.org/search"
52         params = {
53             "q": nombre,
54             "format": "json",
55             "countrycodes": "co",
56             "limit": 1
57         }
58         headers = {"User-Agent": "OptiroutaPlus/1.0"}
59         respuesta = requests.get(url, params=params, headers=headers)
60         resultados = respuesta.json()
61
62         if resultados:
63             r = resultados[0]
64             destinos.append({
65                 "nombre": nombre,
66                 "latitud": float(r["lat"]),
67                 "longitud": float(r["lon"])
68             })
69             print(f" ✓ {nombre}")
70         else:
71             print(f" X No encontrado: {nombre}")
72
73         time.sleep(1)
74
75     print(f"Destinos encontrados: {len(destinos)}")
76     return destinos
```

```

78 # — 3. PUENTE: coordenadas → nodos del grafo (osmnx) —
79 def destinos_a_nodos(G, destinos):
80     print("\nMapeando destinos al grafo de calles...")
81     destinos_con_nodo = []
82
83     for d in destinos:
84         nodo_id = ox.nearest_nodes(G, X=d["longitud"], Y=d["latitud"])
85         data = G.nodes[nodo_id]
86         destinos_con_nodo.append({
87             "nombre": d["nombre"],
88             "latitud_real": d["latitud"],
89             "longitud_real": d["longitud"],
90             "node_id": nodo_id,
91             "latitud_nodo": data["y"],
92             "longitud_nodo": data["x"]
93         })
94         print(f"  ✓ {d['nombre']} → nodo {nodo_id}")
95
96     return destinos_con_nodo
97
98 # — 4. EXTRAER CLIENTES (combina todo) —
99 def extraer_clientes(G, destinos_con_nodo, n=15):
100     print(f"\nGenerando {n} clientes...")
101     nodos = list(G.nodes(data=True))
102     seleccionados = random.sample(nodos, n)
103
104     clientes = []
105     for node_id, data in seleccionados:
106         destino = random.choice(destinos_con_nodo)
107         cliente = {
108             "node_id": node_id,
109             "nombre": f"Cliente_{node_id}",
110             "latitud": data["y"],
111             "longitud": data["x"],
112             "peso_kg": round(random.uniform(1.0, 50.0), 2),
113             "volumen": [
114                 round(random.uniform(10.0, 100.0), 1),
115                 round(random.uniform(10.0, 100.0), 1),
116                 round(random.uniform(10.0, 100.0), 1)

```

```

117         ],
118         "producto": random.choice(PRODUCTOS),
119         "origen": "Centro de distribución Ibagué",
120         "destino": destino["nombre"],
121         "destino_latitud": destino["latitud_real"],
122         "destino_longitud": destino["longitud_real"],
123         "destino_node_id": destino["node_id"]
124     }
125     clientes.append(cliente)
126     print(f" ✓ {cliente['nombre']} → {cliente['destino']}")
127
128     return clientes
129
130 # — 5. MATRIZ DE DISTANCIAS (networkx) —
131 def calcular_distancias(G, clientes):
132     print("\nCalculando matriz de distancias reales...")
133     n = len(clientes)
134     matriz = [[0.0] * n for _ in range(n)]
135
136     for i in range(n):
137         for j in range(n):
138             if i != j:
139                 try:
140                     distancia = nx.shortest_path_length(
141                         G,
142                         clientes[i]["node_id"],
143                         clientes[j]["node_id"],
144                         weight="length"
145                     )
146                     matriz[i][j] = round(distancia, 2)
147                 except nx.NetworkXNoPath:
148                     matriz[i][j] = float("inf")
149
150     print(f"Matriz {n}x{n} calculada")
151     return matriz
152

```

```
# — MAIN —
if __name__ == "__main__":
    G = cargar_grafo()
    destinos = buscar_destinos()
    destinos_con_nodo = destinos_a_nodos(G, destinos)
    clientes = extraer_clientes(G, destinos_con_nodo, n=15)
    matriz = calcular_distancias(G, clientes)

    dataset = {
        "clientes": clientes,
        "destinos_disponibles": destinos_con_nodo,
        "matriz_distancias": matriz
    }

    with open("data/clientes.json", "w", encoding="utf-8") as f:
        json.dump(dataset, f, ensure_ascii=False, indent=2)

    print("\n✓ Dataset completo guardado en data/clientes.json")
```

5.2 Flujo de ejecución.

- **Paso 1** — Extracción: extractor.py descarga el grafo de Ibagué y genera clientes.json con 15 nodos reales y la matriz de distancias 15×15.
- **Paso 2** — Ordenamiento: Insertion Sort ordena los clientes por distancia al depósito.
- **Paso 3** — Voraz: El algoritmo del vecino más cercano genera la ruta inicial y el límite superior para el backtracking.
- **Paso 3B** — Mochila DP: Se asignan los paquetes a cada vehículo maximizando el peso cargado sin superar la capacidad.
- **Paso 4** — Backtracking: Explora el espacio de rutas con poda, encuentra la ruta óptima y registra todos los pasos en JSON.
- **Paso 5** — Exportación: Se genera mapa_data.json con toda la información para la visualización web.

main.py

```
1  import sys
2  import os
3  import time
4  import json
5  | You, la semana pasada * Ultimos cambios
6  # Agregar ruta para imports
7  sys.path.append(os.path.dirname(os.path.abspath(__file__)))
8
9  from algoritmos.voraz import vecino_mas_cercano
10 from algoritmos.mochila_dp import mochila_dp
11 from algoritmos.ordenamiento import insertion_sort
12 from algoritmos.backtracking import backtracking
13
14 def cargar_todo():
15     with open("data/clientes.json", "r", encoding="utf-8") as f:
16         dataset = json.load(f)
17     with open("data/vehiculos.json", "r", encoding="utf-8") as f:
18         vehiculos_data = json.load(f)
19
20     return dataset["clientes"], dataset["matriz_distancias"], vehiculos_data["vehiculos"]
21
22 def mostrar_banner():
23     print("=" * 50)
24     print("      OPTIRUTA+ - Sistema de Ruteo")
25     print("      Ibagué, Tolima - Colombia")
26     print("=" * 50)
27
28 def exportar_mapa_data(clientes, matriz, ruta_voraz, vehiculos, distancia_voraz, asignacion_vehiculos):
29     import json
30
31     # Puntos de clientes con coordenadas reales
32     puntos = []
33     for i, c in enumerate(clientes):
34         puntos.append({
35             "idx": i,
36             "nombre": c["nombre"],
37             "destino": c["destino"],
38             "producto": c["producto"],
```

```

39         "peso_kg": c["peso_kg"],
40         "lat": c["latitud"],
41         "lon": c["longitud"],
42         "destino_lat": c["destino_latitud"],
43         "destino_lon": c["destino_longitud"]
44     })
45
46     # Ruta voraz como secuencia de coordenadas
47     ruta_voraz_coords = []
48     for idx in ruta_voraz:
49         if idx < len(clientes):
50             ruta_voraz_coords.append({
51                 "idx": idx,
52                 "lat": clientes[idx]["latitud"],
53                 "lon": clientes[idx]["longitud"],
54                 "destino": clientes[idx]["destino"]
55             })
56
57     data = {
58         "puntos": puntos,
59         "ruta_voraz": ruta_voraz_coords,
60         "distancia_voraz": distancia_voraz,
61         "vehiculos": [v["tipo"] for v in vehiculos],
62         "asignacion_vehiculos": asignacion_vehiculos,
63         "matriz_distancias": matriz
64     }
65
66     with open("data/mapa_data.json", "w", encoding="utf-8") as f:
67         json.dump(data, f, ensure_ascii=False, indent=2)
68
69     print(" ✓ mapa_data.json generado")
70
71 def main():
72     mostrar_banner()
73     N_CLIENTES = 8
74     # — PASO 1: Cargar datos —————
75     print("\n[1/4] Cargando datos...")
76     clientes, matriz, vehiculos = cargar_todo()

```

```

77     print(f" ✓ {len(clientes)} clientes cargados")
78     print(f" ✓ {len(vehiculos)} vehículos disponibles")
79     print(f" ✓ Matriz de distancias: {len(matriz)}x{len(matriz[0])}")
80
81     # — PASO 2: Ordenar clientes —————
82     print("\n[2/4] Ordenando clientes por distancia al depósito...")
83     distancias = [matriz[0][i] for i in range(len(clientes))]
84     ordenados = insertion_sort(clientes, distancias)
85     print(" Clientes ordenados:")
86     for i, c in enumerate(ordenados):
87         idx = clientes.index(c)
88         print(f"    {i+1}. {c['destino']} - {round(matriz[0][idx], 2)} m")
89
90     # — PASO 3: Ruta voraz —————
91     print("\n[3/4] Calculando ruta inicial (Algoritmo Voraz)...")
92     matriz_voraz = [fila[:N_CLIENTES] for fila in matriz[:N_CLIENTES]]
93     ruta_voraz, distancia_voraz = vecino_mas_cercano(matriz_voraz)
94     print(f" Ruta: {ruta_voraz}")
95     print(f" Distancia total: {distancia_voraz} m ({round(distancia_voraz/1000, 2)} km)")
96     print(f" Complejidad: O(n²) donde n={N_CLIENTES}")
97
98     # — PASO 3B: Mochila DP por vehículo —————
99     print("\n Asignación de paquetes por vehículo (Knapsack DP):")
100    paquetes = [{"nombre": c["producto"], "peso": c["peso_kg"]} for c in clientes[:N_CLIENTES]]
101    asignacion_vehiculos = []
102    for vehiculo in vehiculos:
103        capacidad = vehiculo["peso_max_kg"]
104        peso_total, seleccionados = mochila_dp(capacidad, paquetes)
105        asignacion_vehiculos.append({
106            "tipo": vehiculo["tipo"],
107            "capacidad": capacidad,
108            "peso_cargado": peso_total,
109            "paquetes": seleccionados
110        })
111        print(f" → {vehiculo['tipo']} ({capacidad}kg): {peso_total}kg cargados - {len(seleccionados)} paquetes")
112    # — PASO 4: Backtracking —————
113    print("\n[4/4] Optimizando con Backtracking...")
114    clientes_bt = clientes[:N_CLIENTES]

```

```

115     matriz_bt = [fila[:N_CLIENTES] for fila in matriz[:N_CLIENTES]]
116     pesos = [c["peso_kg"] for c in clientes_bt]
117
118     _, distancia_voraz_bt = vecino_mas_cercano(matriz_bt)
119
120     print(f" Instancia: {N_CLIENTES} clientes")
121     print(f" Límite superior voraz: {distancia_voraz_bt} m")
122
123     ruta_bt_final = []
124     distancia_bt_final = 0.0
125
126     for vehiculo in vehiculos:
127         capacidad = vehiculo["peso_max_kg"]
128         archivo = f"data/pasos_{vehiculo['tipo']}.json"
129
130         inicio = time.time()
131         ruta, distancia = backtracking(matriz_bt, capacidad, pesos, distancia_voraz_bt, archivo)
132         fin = time.time()
133
134         print(f"\n Vehículo: {vehiculo['tipo']} ({capacidad}kg)")
135         if ruta:
136             mejora = round(distancia_voraz_bt - distancia, 2)
137             print(f" Ruta óptima: {ruta}")
138             print(f" Distancia: {distancia} m - Mejora: {mejora} m")
139             if not ruta_bt_final: # guarda la primera ruta factible
140                 ruta_bt_final = ruta
141                 distancia_bt_final = distancia
142         else:
143             print(f" Sin ruta factible - Voraz: {distancia_voraz_bt} m")
144             print(f" Tiempo: {round(fin - inicio, 4)}s")
145
146     # --- EXPORTAR DATOS PARA MAPA ---
147     print("\nExportando datos para mapa interactivo...")
148     exportar_mapa_data(clientes[:N_CLIENTES], matriz_voraz, ruta_voraz, vehiculos, distancia_voraz, asignacion_vehiculos)
149
150     # --- RESUMEN FINAL ---
151     print("\n" + "=" * 50)
152     print(" RESUMEN COMPARATIVO")

```

```

150     # --- RESUMEN FINAL ---
151     print("\n" + "=" * 50)
152     print(" RESUMEN COMPARATIVO")
153     print("=" * 50)
154     print(f" Clientes:      {N_CLIENTES}")
155     print(f" Voraz:           {distancia_voraz} m - O(n²)")
156     if distancia_bt_final > 0:
157         print(f" Backtracking:   {distancia_bt_final} m - O(n!) podado")
158         print(f" Mejora:         {round(distancia_voraz - distancia_bt_final, 2)} m")
159     else:
160         print(f" Backtracking:   Sin ruta factible")
161     print("=" * 50)
162     print("\n✓ Archivos de pasos generados en data/")
163     print("✓ Abre visualizacion/mapa.html para ver el mapa interactivo")
164     print("✓ Abre visualizacion/index.html para ver el árbol de backtracking")
165
166     if __name__ == "__main__":
167         main()

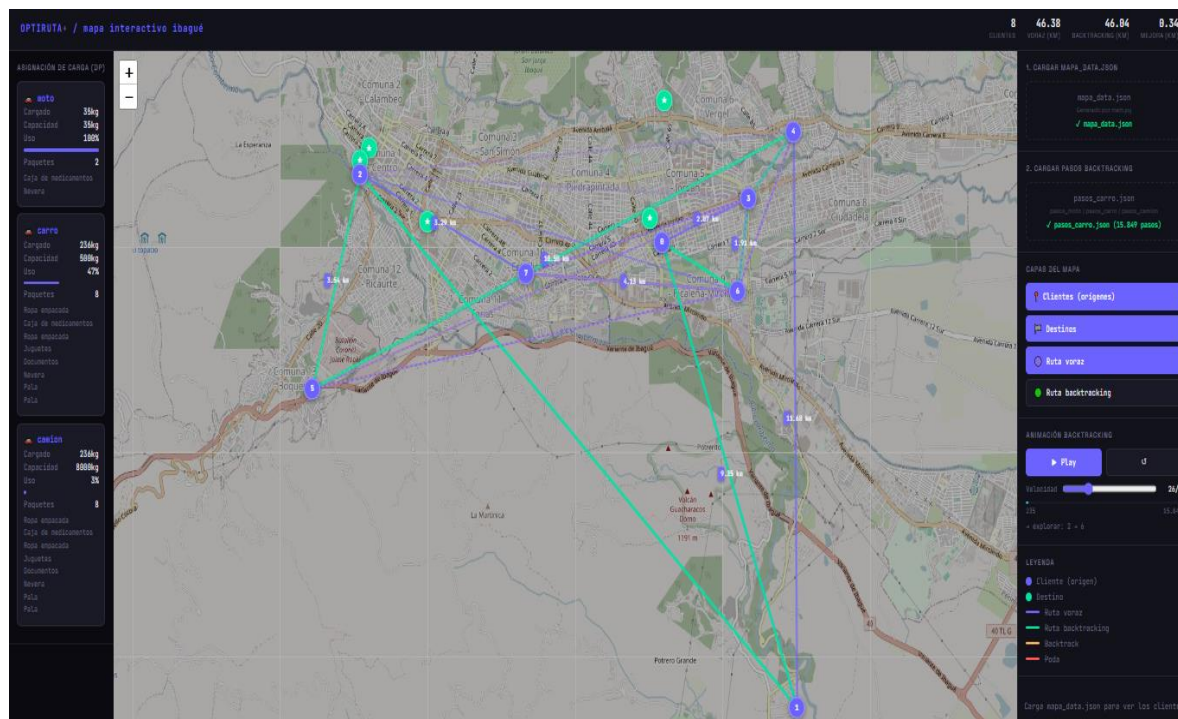
```

6. VISUALIZACIÓN INTERACTIVA

6.1 Mapa interactivo (mapa.html).

Se desarrolló un mapa interactivo usando Leaflet.js con tiles de OpenStreetMap que muestra sobre el mapa real de Ibagué:

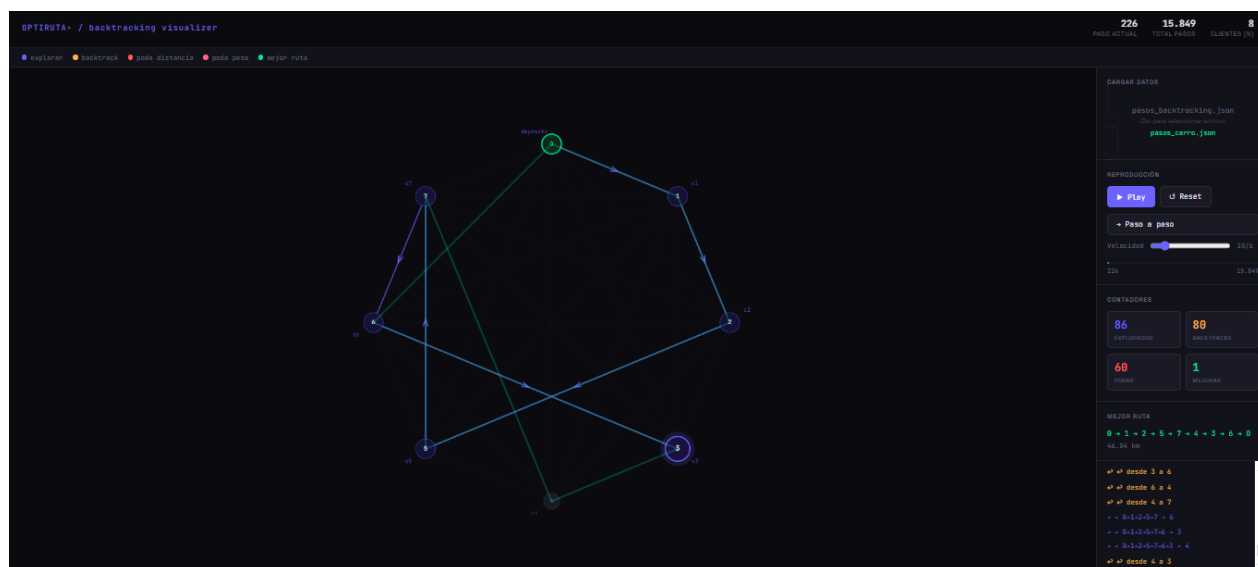
- Marcadores de los 8 clientes (orígenes) con información detallada al hacer clic: producto, peso y destino.
- Marcadores de los destinos reales (centros comerciales, hospitales, universidades, etc.).
- Ruta voraz dibujada en color morado con etiquetas de distancia en km entre cada par de nodos.
- Animación del proceso de backtracking: exploración en azul, retrocesos en amarillo y podas en rojo.
- Panel lateral con la asignación de carga por vehículo mostrando peso cargado, capacidad y porcentaje de uso.
- Estadísticas en tiempo real: número de clientes, distancia voraz, distancia backtracking y mejora obtenida.



6.2 Árbol de backtracking (index.html).

Se implementó una segunda visualización que muestra el árbol de estados del backtracking usando Canvas:

- Los nodos se distribuyen en un anillo circular con el depósito marcado como D.
- Las aristas muestran en tiempo real qué nodo se está explorando, con colores diferenciados por tipo de paso.
- Contadores de explorados, backtracks, podas y mejoras actualizados en tiempo real.
- Control de velocidad ajustable de 1 a 200 pasos por segundo.
- Modo paso a paso para análisis detallado del algoritmo.



CONCLUSION

El proyecto OPTIRUTA+ demostró de manera práctica la utilidad de combinar múltiples paradigmas algorítmicos para resolver un problema real de logística:

- El Algoritmo Voraz genera soluciones rápidas y aceptables en $O(n^2)$, siendo ideal cuando se necesita una respuesta inmediata. Sin embargo, no garantiza la optimalidad — en nuestro caso, dejó 6,17 km de margen de mejora.
- La Programación Dinámica permite resolver de forma exacta el subproblema de asignación de carga en tiempo pseudo-polinomial, siendo la herramienta correcta para este tipo de problema de optimización con restricciones de capacidad.
- El Backtracking con poda es el único algoritmo que garantiza encontrar la solución exacta al problema de ruteo. La poda por distancia y por capacidad reduce drásticamente el espacio de búsqueda, haciendo viable su uso para instancias pequeñas ($N \leq 12$). Para $N=15$ el tiempo supera los 2 minutos, lo que evidencia la naturaleza NP-difícil del problema.
- El Insertion Sort, aunque sencillo, cumple su papel de ordenar los clientes de forma transparente y eficiente para instancias pequeñas, demostrando que la elección del algoritmo debe estar guiada por el contexto del problema.
- El uso de datos geográficos reales de Ibagué (OpenStreetMap, osmnx, Nominatim) agrega valor al proyecto al demostrar que los algoritmos estudiados son aplicables a problemas del mundo real, no solo a instancias teóricas.

REFERENCIAS

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.
- Boeing, G. (2017). OSMnx: New Methods for Acquiring, Constructing, Analyzing, and Visualizing Complex Street Networks. Computers, Environment and Urban Systems, 65, 126-139.
- OpenStreetMap contributors. (2024). OpenStreetMap. <https://www.openstreetmap.org>
- NetworkX developers. (2024). NetworkX: Network Analysis in Python. <https://networkx.org>
- Leaflet.js contributors. (2024). Leaflet: a JavaScript library for mobile-friendly interactive maps. <https://leafletjs.com>
- Toth, P., & Vigo, D. (Eds.). (2014). Vehicle Routing: Problems, Methods, and Applications (2nd ed.). Society for Industrial and Applied Mathematics.